// app/api/auth/login/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import bcrypt from 'bcryptjs'
import pool from '@/lib/db'
import { signToken } from '@/lib/auth'
import { success, failure } from '@/lib/result'

export async function POST(req: NextRequest) {
  const body = await req.json()
  const { login, password } = body as { login?: string; password?: string }

  if (!login || !password) {
    return NextResponse.json(failure('username/email and password are required', 400), { status: 400 })
  }

  const result = await pool.query(
    'SELECT id, username, email, password_hash FROM users WHERE email = $1 OR username = $1',
    [login]
  )

  if (result.rows.length === 0) {
    return NextResponse.json(failure('Invalid credentials', 401), { status: 401 })
  }

  const user = result.rows[0]
  const valid = await bcrypt.compare(password, user.password_hash)

  if (!valid) {
    return NextResponse.json(failure('Invalid credentials', 401), { status: 401 })
  }

  const token = signToken({ userId: user.id, username: user.username })
  const { password_hash: _, ...userWithoutHash } = user

  const response = NextResponse.json(success({ user: userWithoutHash, token }))
  response.cookies.set('token', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: 60 * 60 * 24 * 7,
  })
  return response
}

```

// app/api/auth/logout/route.ts

```
import { NextResponse } from 'next/server'
import { success } from '@/lib/result'

export async function POST() {
  const response = NextResponse.json(success({ loggedOut: true }))
  response.cookies.delete('token')
  return response
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

```
}

```

// app/api/auth/register/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import bcrypt from 'bcryptjs'
import pool from '@/lib/db'
import { signToken } from '@/lib/auth'
import { success, failure } from '@/lib/result'

export async function POST(req: NextRequest) {
  const body = await req.json()
  const { username, email, password } = body as {
    username?: string
    email?: string
    password?: string
  }

  if (!username || !email || !password) {
    return NextResponse.json(failure('username, email and password are required', 400), { status: 400 })
  }

  const passwordHash = await bcrypt.hash(password, 10)

  try {
    const result = await pool.query(
      'INSERT INTO users (username, email, password_hash) VALUES ($1, $2, $3) RETURNING id, username, email, created_at',
      [username, email, passwordHash]
    )
    const user = result.rows[0]
    const token = signToken({ userId: user.id, username: user.username })

    const response = NextResponse.json(success({ user, token }), { status: 201 })
    response.cookies.set('token', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'lax',
      maxAge: 60 * 60 * 24 * 7,
    })
    return response
  } catch (err: unknown) {
    const pgErr = err as { code?: string }
    if (pgErr.code === '23505') {
      return NextResponse.json(failure('Username or email already exists', 409), { status: 409 })
    }
    console.error(err)
    return NextResponse.json(failure('Server error', 500), { status: 500 })
  }
}

```

// app/api/dependencies/[projectId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

```
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ projectId: string }> }

async function verifyOwnership(projectId: string, userId: string) {
  const r = await pool.query(
    'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
    [projectId, userId]
  )
  return r.rows.length > 0
}

export async function POST(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: 401 })
  const { projectId } = await params

  if (!(await verifyOwnership(projectId, auth.value.userId)))
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const { from_task_id, to_task_id, type = 2, lag = 0 } = await req.json()
  if (!from_task_id || !to_task_id)
    return NextResponse.json(failure('from_task_id and to_task_id required', 400), { status: 400 })

  const r = await pool.query(
    `INSERT INTO dependencies (project_id, from_task_id, to_task_id, type, lag)
     VALUES ($1,$2,$3,$4,$5) RETURNING *`,
    [projectId, from_task_id, to_task_id, type, lag]
  )
  return NextResponse.json(success(r.rows[0]), { status: 201 })
}

export async function PUT(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: 401 })
  const { projectId } = await params

  if (!(await verifyOwnership(projectId, auth.value.userId)))
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const { id, type, lag } = await req.json()
  if (!id) return NextResponse.json(failure('id required', 400), { status: 400 })

  const r = await pool.query(
    `UPDATE dependencies
     SET type = COALESCE($1, type), lag = COALESCE($2, lag)
     WHERE id = $3 AND project_id = $4 RETURNING *`,
    [type ?? null, lag ?? null, id, projectId]
  )
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

```
  if (!r.rows[0]) return NextResponse.json(failure('Not found', 404), { status: 404 })
  return NextResponse.json(success(r.rows[0]))
}

export async function DELETE(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: 401 })
  const { projectId } = await params

  if (!(await verifyOwnership(projectId, auth.value.userId)))
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const { id } = await req.json()
  if (!id) return NextResponse.json(failure('id required', 400), { status: 400 })

  await pool.query(
    'DELETE FROM dependencies WHERE id = $1 AND project_id = $2',
    [id, projectId]
  )
  return NextResponse.json(success({ deleted: id }))
}

```

// app/api/projects/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ id: string }> }

export async function GET(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { id } = await params

  const result = await pool.query(
    'SELECT * FROM projects WHERE id = $1 AND user_id = $2',
    [id, auth.value.userId]
  )
  if (result.rows.length === 0) {
    return NextResponse.json(failure('Project not found', 404), { status: 404 })
  }
  return NextResponse.json(success(result.rows[0]))
}

export async function PUT(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { id } = await params

  const body = await req.json()
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

```
  const { name, start_date, end_date, status_date } = body as {
    name?: string
    start_date?: string | null
    end_date?: string | null
    status_date?: string | null
  }

  const result = await pool.query(
    `UPDATE projects
     SET name = COALESCE($1, name),
         start_date = COALESCE($2, start_date),
         end_date = COALESCE($3, end_date),
         status_date = $4
     WHERE id = $5 AND user_id = $6
     RETURNING *`,
    [name ?? null, start_date ?? null, end_date ?? null, status_date ?? null, id, auth.value.userId]
  )
  if (result.rows.length === 0) {
    return NextResponse.json(failure('Project not found', 404), { status: 404 })
  }
  return NextResponse.json(success(result.rows[0]))
}

export async function DELETE(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { id } = await params

  const result = await pool.query(
    'DELETE FROM projects WHERE id = $1 AND user_id = $2 RETURNING id',
    [id, auth.value.userId]
  )
  if (result.rows.length === 0) {
    return NextResponse.json(failure('Project not found', 404), { status: 404 })
  }
  return NextResponse.json(success({ deleted: true }))
}

```

// app/api/projects/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

export async function GET(req: NextRequest) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const result = await pool.query(
    'SELECT * FROM projects WHERE user_id = $1 ORDER BY created_at DESC',
    [auth.value.userId]
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

```
  )
  return NextResponse.json(success(result.rows))
}

export async function POST(req: NextRequest) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const body = await req.json()
  const { name, start_date, end_date, status_date } = body as {
    name?: string
    start_date?: string
    end_date?: string
    status_date?: string
  }

  if (!name) {
    return NextResponse.json(failure('name is required', 400), { status: 400 })
  }

  const result = await pool.query(
    'INSERT INTO projects (user_id, name, start_date, end_date, status_date) VALUES ($1, $2, $3, $4, $5) RETURNING *',
    [auth.value.userId, name, start_date ?? null, end_date ?? null, status_date ?? null]
  )
  return NextResponse.json(success(result.rows[0]), { status: 201 })
}

```

// app/api/tasks/[projectId]/[taskId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ projectId: string; taskId: string }> }

// GET /api/tasks/[projectId]/[taskId] — task detail + lifecycle events
export async function GET(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId, taskId } = await params

  const proj = await pool.query(
    'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
    [projectId, auth.value.userId]
  )
  if (!proj.rows.length)
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const [taskRes, lifecycleRes] = await Promise.all([
    pool.query('SELECT * FROM tasks WHERE id = $1 AND project_id = $2', [taskId, projectId]),
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

```
    pool.query(
      `SELECT l.*, u.username AS created_by_name
       FROM task_lifecycle l
       LEFT JOIN users u ON u.id = l.created_by
       WHERE l.task_id = $1
       ORDER BY l.created_at DESC`,
      [taskId]
    ),
  ])

  if (!taskRes.rows.length)
    return NextResponse.json(failure('Task not found', 404), { status: 404 })

  return NextResponse.json(success({
    task: taskRes.rows[0],
    lifecycle: lifecycleRes.rows,
  }))
}

```

// app/api/tasks/[projectId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import type { PoolClient } from 'pg'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ projectId: string }> }

async function verifyProjectOwnership(projectId: string, userId: string): Promise<boolean> {
  const result = await pool.query(
    'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
    [projectId, userId]
  )
  return result.rows.length > 0
}

// Generate next task_code for a project: T-001, T-002, ...
async function nextTaskCode(client: PoolClient, projectId: string): Promise<string> {
  const res = await client.query(
    `SELECT COALESCE(MAX(CAST(SUBSTRING(task_code FROM 3) AS INTEGER)), 0) + 1 AS seq
     FROM tasks WHERE project_id = $1 AND task_code IS NOT NULL`,
    [projectId]
  )
  const seq = res.rows[0].seq as number
  return `T-${String(seq).padStart(3, '0')}`
}

// Insert a lifecycle event
async function addLifecycle(
  client: PoolClient,
  opts: {
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
    taskId: string; taskCode: string; projectId: string; userId: string
    event_type: string; field_name?: string | null
    old_value?: string | null; new_value?: string | null
    description: string
  }
) {
  await client.query(
    `INSERT INTO task_lifecycle
       (task_id, task_code, project_id, event_type, field_name, old_value, new_value, description, created_by)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9)`,
    [opts.taskId, opts.taskCode, opts.projectId, opts.event_type,
     opts.field_name ?? null, opts.old_value ?? null, opts.new_value ?? null,
     opts.description, opts.userId]
  )
}

const FIELD_LABELS: Record<string, string> = {
  name: '任务名称', start_date: '开始日期', end_date: '结束日期',
  duration: '工期', assignee: '责任人', percent_done: '完成度',
  parent_id: '父任务', is_milestone: '里程碑', note: '备注',
}

function fmtFieldVal(field: string, val: unknown): string {
  if (val === null || val === undefined || val === '') return '(空)'
  if (field === 'start_date' || field === 'end_date') {
    const s = String(val); return s.includes('T') ? s.split('T')[0] : s
  }
  if (field === 'duration') return `${val} 天`
  if (field === 'percent_done') return `${val}%`
  if (field === 'is_milestone') return val ? '是' : '否'
  return String(val)
}

// ── GET ──────────────────────────────────────────────────────────────
export async function GET(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const [tasks, deps] = await Promise.all([
    pool.query(
      'SELECT * FROM tasks WHERE project_id = $1 AND is_deleted = false ORDER BY order_index ASC',
      [projectId]
    ),
    // Exclude deps where either endpoint is soft-deleted
    pool.query(
      `SELECT d.* FROM dependencies d
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
        JOIN tasks ft ON ft.id = d.from_task_id AND ft.is_deleted = false
        JOIN tasks tt ON tt.id = d.to_task_id   AND tt.is_deleted = false
        WHERE d.project_id = $1`,
       [projectId]
      ),
    ])

  return NextResponse.json(success({ tasks: tasks.rows, dependencies: deps.rows }))
}

// ── POST ──────────────────────────────────────────────────────────────
export async function POST(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const body = await req.json()
  const taskInputs = Array.isArray(body) ? body : [body]

  const client = await pool.connect()
  const inserted = []
  try {
    await client.query('BEGIN')
    for (const task of taskInputs) {
      const { name, parent_id, assignee, start_date, end_date, duration, duration_unit,
              percent_done, is_milestone, note, order_index } = task as Record<string, unknown>
      if (!name) continue

      const code = await nextTaskCode(client, projectId)

      const r = await client.query(
        `INSERT INTO tasks
           (project_id, parent_id, task_code, name, assignee, start_date, end_date,
            duration, duration_unit, percent_done, is_milestone, note, order_index)
         VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13) RETURNING *`,
        [projectId, parent_id ?? null, code, name,
         assignee ?? null, start_date ?? null, end_date ?? null,
         duration ?? null, duration_unit ?? 'day',
         percent_done ?? 0, is_milestone ?? false, note ?? null, order_index ?? 0]
      )
      const newTask = r.rows[0]
      inserted.push(newTask)

      await addLifecycle(client, {
        taskId: newTask.id, taskCode: code, projectId,
        userId: auth.value.userId, event_type: 'created',
        description: `创建任务「${name}」（${code}）`,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
      })
    }
    await client.query('COMMIT')
  } catch (err) {
    await client.query('ROLLBACK')
    throw err
  } finally {
    client.release()
  }

  return NextResponse.json(success(inserted), { status: 201 })
}

// ── PUT ──────────────────────────────────────────────────────────────
export async function PUT(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const body = await req.json()
  const taskInputs = Array.isArray(body) ? body : [body]

  const client = await pool.connect()
  const updated = []
  try {
    await client.query('BEGIN')

    for (const task of taskInputs) {
      const { id } = task as { id: string }
      if (!id) continue

      // Fetch current state for diff
      const prev = await client.query(
        'SELECT * FROM tasks WHERE id = $1 AND project_id = $2 AND is_deleted = false',
        [id, projectId]
      )
      if (!prev.rows.length) continue
      const old = prev.rows[0]

      const hasParentKey = 'parent_id' in task
      const hasAssigneeKey = 'assignee' in task

      const r = await client.query(
        `UPDATE tasks SET
           name         = COALESCE($1, name),
           parent_id    = CASE WHEN $13 THEN $2 ELSE parent_id END,
           assignee     = CASE WHEN $14 THEN $3 ELSE assignee END,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
            start_date   = COALESCE($4, start_date),
            end_date     = COALESCE($5, end_date),
            duration     = COALESCE($6, duration),
            duration_unit= COALESCE($7, duration_unit),
            percent_done = COALESCE($8, percent_done),
            is_milestone = COALESCE($9, is_milestone),
            note         = $10,
            order_index  = COALESCE($11, order_index),
            updated_at   = NOW()
          WHERE id = $12 AND project_id = $15
          RETURNING *`,
         [
           (task as Record<string, unknown>).name ?? null,
           (task as Record<string, unknown>).parent_id ?? null,
           (task as Record<string, unknown>).assignee ?? null,
           (task as Record<string, unknown>).start_date ?? null,
           (task as Record<string, unknown>).end_date ?? null,
           (task as Record<string, unknown>).duration ?? null,
           (task as Record<string, unknown>).duration_unit ?? null,
           (task as Record<string, unknown>).percent_done ?? null,
           (task as Record<string, unknown>).is_milestone ?? null,
           (task as Record<string, unknown>).note ?? null,
           (task as Record<string, unknown>).order_index ?? null,
           id, hasParentKey, hasAssigneeKey, projectId,
         ]
       )
       if (!r.rows[0]) continue
       const cur = r.rows[0]
       updated.push(cur)

       const code: string = cur.task_code ?? old.task_code ?? id.slice(0, 8)

       // ── Detect position move (order_index or parent_id changed) ────────
       const parentChanged = hasParentKey && String(old.parent_id ?? '') !== String(cur.parent_id ?? '')
       const orderChanged  = (task as Record<string, unknown>).order_index !== undefined
                           && old.order_index !== cur.order_index

       if (parentChanged || orderChanged) {
         let desc: string
         if (parentChanged && cur.parent_id === null) {
           desc = `任务「${cur.name}」（${code}）升级为顶级任务`
         } else if (parentChanged) {
           // look up new parent name
           const parentRes = await client.query('SELECT name, task_code FROM tasks WHERE id = $1', [cur.parent_id])
           const pName = parentRes.rows[0]?.name ?? cur.parent_id
           const pCode = parentRes.rows[0]?.task_code ?? ''
           desc = `任务「${cur.name}」（${code}）移至子任务，父级：「${pName}」（${pCode}）`
         } else {
           const dir = cur.order_index < old.order_index ? '上移' : '下移'
           desc = `任务「${cur.name}」（${code}）${dir}（排序 ${old.order_index} → ${cur.order_index}）`
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
          }
          await addLifecycle(client, {
            taskId: id, taskCode: code, projectId, userId: auth.value.userId,
            event_type: 'moved',
            field_name: parentChanged ? 'parent_id' : 'order_index',
            old_value: parentChanged ? String(old.parent_id ?? '顶级') : String(old.order_index),
            new_value: parentChanged ? String(cur.parent_id ?? '顶级') : String(cur.order_index),
            description: desc,
          })
        }

        // ── Detect field updates ──────────────────────────────────────
        const COMPARE_FIELDS = ['name','start_date','end_date','duration','assignee','percent_done','is_milestone','note']
        for (const f of COMPARE_FIELDS) {
          if (!(f in task)) continue
          const norm = (v: unknown) => {
            if (v === null || v === undefined) return ''
            const s = String(v); return s.includes('T') ? s.split('T')[0] : s
          }
          const ov = norm(old[f])
          const nv = norm(cur[f])
          if (ov === nv) continue
          const label = FIELD_LABELS[f] ?? f
          const ovStr = fmtFieldVal(f, old[f])
          const nvStr = fmtFieldVal(f, cur[f])
          let desc: string
          if (f === 'duration') {
            const diff = Number(cur[f] ?? 0) - Number(old[f] ?? 0)
            desc = diff > 0
              ? `任务「${cur.name}」（${code}）工期延长 ${diff} 天（${ovStr} → ${nvStr}）`
              : `任务「${cur.name}」（${code}）工期缩短 ${Math.abs(diff)} 天（${ovStr} → ${nvStr}）`
          } else if (f === 'name') {
            desc = `任务（${code}）重命名：「${ovStr}」→「${nvStr}」`
          } else {
            desc = `任务「${cur.name}」（${code}）${label}: ${ovStr} → ${nvStr}`
          }
          await addLifecycle(client, {
            taskId: id, taskCode: code, projectId, userId: auth.value.userId,
            event_type: 'updated', field_name: f,
            old_value: ovStr, new_value: nvStr, description: desc,
          })
        }
      }

      await client.query('COMMIT')
    } catch (err) {
      await client.query('ROLLBACK')
      throw err
    } finally {
      client.release()
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
  }

  return NextResponse.json(success(updated))
}

// ── DELETE (soft) ──────────────────────────────────────────────────────
export async function DELETE(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const body = await req.json()
  const ids: string[] = Array.isArray(body.ids) ? body.ids : [body.id]
  if (!ids.length) return NextResponse.json(failure('No IDs provided', 400), { status: 400 })

  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    // Collect all descendants recursively for cascade soft-delete
    const allIds = new Set<string>(ids)
    let frontier = [...ids]
    while (frontier.length) {
      const ph = frontier.map((_, i) => `$${i + 1}`).join(',')
      const children = await client.query(
        `SELECT id FROM tasks WHERE parent_id IN (${ph}) AND is_deleted = false`,
        frontier
      )
      const childIds = children.rows.map((r: { id: string }) => r.id).filter((id: string) => !allIds.has(id))
      childIds.forEach((id: string) => allIds.add(id))
      frontier = childIds
    }

    const allIdArr = [...allIds]
    const ph = allIdArr.map((_, i) => `$${i + 2}`).join(',')

    // Fetch tasks for lifecycle before soft-deleting
    const toDelete = await client.query(
      `SELECT id, task_code, name FROM tasks WHERE project_id = $1 AND id IN (${ph})`,
      [projectId, ...allIdArr]
    )

    // Soft-delete
    await client.query(
      `UPDATE tasks SET is_deleted = true, deleted_at = NOW()
       WHERE project_id = $1 AND id IN (${ph})`,
      [projectId, ...allIdArr]
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
    )

    // Lifecycle events
    for (const t of toDelete.rows) {
      await addLifecycle(client, {
        taskId: t.id, taskCode: t.task_code ?? t.id.slice(0, 8),
        projectId, userId: auth.value.userId,
        event_type: 'deleted',
        description: `删除任务「${t.name}」（${t.task_code ?? ''}）`,
      })
    }

    await client.query('COMMIT')
    return NextResponse.json(success({ deleted: allIdArr }))
  } catch (err) {
    await client.query('ROLLBACK')
    throw err
  } finally {
    client.release()
  }
}

```

// app/api/versions/[projectId]/[versionId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'

type Params = { projectId: string; versionId: string }

// GET /api/versions/[projectId]/[versionId] — get snapshot data
export async function GET(req: NextRequest, { params }: { params: Promise<Params> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId, versionId } = await params
  const client = await pool.connect()
  try {
    const res = await client.query(
      `SELECT v.* FROM project_versions v
       JOIN projects p ON p.id = v.project_id
       WHERE v.id = $1 AND v.project_id = $2 AND p.user_id = $3`,
      [versionId, projectId, auth.value.userId],
    )
    if (!res.rows.length)
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })

    return NextResponse.json({ ok: true, value: res.rows[0].snapshot })
  } finally {
    client.release()
  }
}
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

```

// POST /api/versions/[projectId]/[versionId]/restore — restore version
export async function POST(req: NextRequest, { params }: { params: Promise<Params> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId, versionId } = await params
  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    const res = await client.query(
      `SELECT v.snapshot FROM project_versions v
       JOIN projects p ON p.id = v.project_id
       WHERE v.id = $1 AND v.project_id = $2 AND p.user_id = $3`,
      [versionId, projectId, auth.value.userId],
    )
    if (!res.rows.length) {
      await client.query('ROLLBACK')
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })
    }

    const { tasks, dependencies } = res.rows[0].snapshot as { tasks: Record<string, unknown>[]; dependencies: Record<string,

    // Wipe current tasks + deps (cascade removes deps too)
    await client.query('DELETE FROM tasks WHERE project_id = $1', [projectId])

    // Re-insert tasks (two passes: parents first, then children)
    const sorted = [...tasks].sort((a, b) => {
      if (!a.parent_id && b.parent_id) return -1
      if (a.parent_id && !b.parent_id) return 1
      return 0
    })
    for (const t of sorted) {
      await client.query(
        `INSERT INTO tasks
           (id, project_id, parent_id, name, assignee, start_date, end_date,
            duration, duration_unit, percent_done, is_milestone, note, order_index,
            created_at, updated_at)
         VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15)
         ON CONFLICT (id) DO UPDATE SET
           name=EXCLUDED.name, assignee=EXCLUDED.assignee,
           start_date=EXCLUDED.start_date, end_date=EXCLUDED.end_date,
           duration=EXCLUDED.duration, duration_unit=EXCLUDED.duration_unit,
           percent_done=EXCLUDED.percent_done, is_milestone=EXCLUDED.is_milestone,
           note=EXCLUDED.note, order_index=EXCLUDED.order_index,
           updated_at=NOW()`,
        [
          t.id, projectId, t.parent_id ?? null, t.name, t.assignee ?? null,
          t.start_date ?? null, t.end_date ?? null,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

```
          t.duration ?? null, t.duration_unit ?? 'day',
          t.percent_done ?? 0, t.is_milestone ?? false,
          t.note ?? null, t.order_index ?? 0,
          t.created_at, t.updated_at,
        ],
      )
    }

    // Re-insert dependencies
    for (const d of dependencies) {
      await client.query(
        `INSERT INTO dependencies (id, project_id, from_task_id, to_task_id, type, lag)
         VALUES ($1,$2,$3,$4,$5,$6)
         ON CONFLICT (id) DO NOTHING`,
        [d.id, projectId, d.from_task_id, d.to_task_id, d.type ?? 2, d.lag ?? 0],
      )
    }

    await client.query('COMMIT')
    return NextResponse.json({ ok: true, value: { tasks, dependencies } })
  } catch (err) {
    await client.query('ROLLBACK')
    console.error('Restore failed', err)
    return NextResponse.json({ ok: false, error: 'Restore failed' }, { status: 500 })
  } finally {
    client.release()
  }
}

```

// app/api/versions/[projectId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'

// ── Human-readable field names ──────────────────────────────────
const FIELD_LABELS: Record<string, string> = {
  name:         '任务名称',
  start_date:   '开始日期',
  end_date:     '结束日期',
  duration:     '工期',
  assignee:     '责任人',
  percent_done: '完成度',
  parent_id:    '层级',
  is_milestone: '里程碑',
  note:         '备注',
}

function fmtVal(field: string, val: unknown): string {
  if (val === null || val === undefined || val === '') return '(空)'
  if (field === 'start_date' || field === 'end_date') {
    const s = String(val)
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
    return s.includes('T') ? s.split('T')[0] : s
  }
  if (field === 'duration') return `${val} 天`
  if (field === 'percent_done') return `${val}%`
  if (field === 'is_milestone') return val ? '是' : '否'
  return String(val)
}

type TaskRow = Record<string, unknown>
type DepRow  = Record<string, unknown>

interface ChangeEntry {
  change_type: string
  task_name:   string | null
  field_name:  string | null
  old_value:   string | null
  new_value:   string | null
  description: string
}

// Compute depth-first flat order (same as client-side), returns ordered task IDs
function getFlatOrder(tasks: TaskRow[]): string[] {
  const kids: Record<string, TaskRow[]> = {}
  for (const t of tasks) {
    const k = (t.parent_id ?? '__root__') as string
    if (!kids[k]) kids[k] = []
    kids[k].push(t)
  }
  const order: string[] = []
  function walk(pid: string | null) {
    const key = pid ?? '__root__'
    ;(kids[key] ?? [])
      .sort((a, b) => (a.order_index as number) - (b.order_index as number))
      .forEach(t => { order.push(t.id as string); walk(t.id as string) })
  }
  walk(null)
  return order
}

function diffSnapshots(
  prevTasks: TaskRow[], prevDeps: DepRow[],
  currTasks: TaskRow[], currDeps: DepRow[],
): ChangeEntry[] {
  const changes: ChangeEntry[] = []

  const prevMap = new Map(prevTasks.map(t => [t.id as string, t]))
  const currMap = new Map(currTasks.map(t => [t.id as string, t]))

  // Added tasks
  for (const t of currTasks) {
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
      if (!prevMap.has(t.id as string)) {
        changes.push({
          change_type: 'task_add',
          task_name:   t.name as string,
          field_name:  null, old_value: null, new_value: null,
          description: `新增任务「${t.name}」`,
        })
      }
    }

    // Deleted tasks
    for (const t of prevTasks) {
      if (!currMap.has(t.id as string)) {
        changes.push({
          change_type: 'task_delete',
          task_name:   t.name as string,
          field_name:  null, old_value: null, new_value: null,
          description: `删除任务「${t.name}」`,
        })
      }
    }

    // Updated tasks
    const COMPARE_FIELDS = ['name','start_date','end_date','duration','assignee','percent_done','parent_id','is_milestone','no
    for (const curr of currTasks) {
      const prev = prevMap.get(curr.id as string)
      if (!prev) continue
      for (const f of COMPARE_FIELDS) {
        const pv = prev[f] ?? null
        const cv = curr[f] ?? null
        // Normalise dates to YYYY-MM-DD for comparison
        const norm = (v: unknown) => {
          if (!v) return null
          const s = String(v)
          return s.includes('T') ? s.split('T')[0] : s
        }
        const pvN = (f === 'start_date' || f === 'end_date') ? norm(pv) : pv
        const cvN = (f === 'start_date' || f === 'end_date') ? norm(cv) : cv
        if (String(pvN ?? '') !== String(cvN ?? '')) {
          const label = FIELD_LABELS[f] ?? f
          const pStr  = fmtVal(f, pv)
          const cStr  = fmtVal(f, cv)
          let desc: string
          if (f === 'duration') {
            const diff = Number(cv ?? 0) - Number(pv ?? 0)
            desc = diff > 0
              ? `任务「${curr.name}」工期延长 ${diff} 天（${pStr} → ${cStr}）`
              : `任务「${curr.name}」工期缩短 ${Math.abs(diff)} 天（${pStr} → ${cStr}）`
          } else if (f === 'start_date') {
            desc = `任务「${curr.name}」开始日期从 ${pStr} 改为 ${cStr}`
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
          } else if (f === 'end_date') {
            desc = `任务「${curr.name}」结束日期从 ${pStr} 改为 ${cStr}`
          } else if (f === 'name') {
            desc = `任务重命名: 「${pStr}」→「${cStr}」`
          } else if (f === 'parent_id') {
            desc = cv
              ? `任务「${curr.name}」设为子任务`
              : `任务「${curr.name}」升级为顶级任务`
          } else {
            desc = `任务「${curr.name}」${label}: ${pStr} → ${cStr}`
          }
          changes.push({
            change_type: 'task_update',
            task_name:   curr.name as string,
            field_name:  f,
            old_value:   pStr,
            new_value:   cStr,
            description: desc,
          })
        }
      }
    }

    // ── Position changes (reorder / swap) ──────────────────────────────
    // Only consider tasks present in both versions; ignore added/deleted tasks here
    const bothIds = new Set([...prevMap.keys()].filter(id => currMap.has(id)))

    // Compute flat tree row number restricted to shared tasks only
    // (filtering out added/deleted keeps numbering stable across the two snapshots)
    const prevShared = getFlatOrder(prevTasks).filter(id => bothIds.has(id))
    const currShared = getFlatOrder(currTasks).filter(id => bothIds.has(id))

    const prevPos: Record<string, number> = {}
    const currPos: Record<string, number> = {}
    prevShared.forEach((id, i) => { prevPos[id] = i + 1 })
    currShared.forEach((id, i) => { currPos[id] = i + 1 })

    const moved = [...bothIds].filter(id => prevPos[id] !== currPos[id])

    // Detect pure swaps: A was at p→q, B was at q→p
    const swapped = new Set<string>()
    for (let i = 0; i < moved.length; i++) {
      const idA = moved[i]
      if (swapped.has(idA)) continue
      for (let j = i + 1; j < moved.length; j++) {
        const idB = moved[j]
        if (swapped.has(idB)) continue
        if (prevPos[idA] === currPos[idB] && prevPos[idB] === currPos[idA]) {
          const nameA = currMap.get(idA)!.name as string
          const nameB = currMap.get(idB)!.name as string
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
        changes.push({
          change_type: 'task_reorder',
          task_name:   nameA,
          field_name:  null,
          old_value:   String(prevPos[idA]),
          new_value:   String(currPos[idA]),
          description: `任务「${nameA}」与「${nameB}」交换了位置（第${prevPos[idA]}行 ⇔ 第${prevPos[idB]}行）`,
        })
        swapped.add(idA)
        swapped.add(idB)
        break
      }
    }
  }

  // Remaining non-swap moves
  for (const id of moved) {
    if (swapped.has(id)) continue
    const name = currMap.get(id)!.name as string
    const from = prevPos[id]
    const to   = currPos[id]
    changes.push({
      change_type: 'task_reorder',
      task_name:   name,
      field_name:  null,
      old_value:   String(from),
      new_value:   String(to),
      description: `任务「${name}」${to < from ? '上移' : '下移'}（第${from}行 → 第${to}行）`,
    })
  }

  // Dependencies
  const prevDepSet = new Set(prevDeps.map(d => `${d.from_task_id}→${d.to_task_id}`))
  const currDepSet = new Set(currDeps.map(d => `${d.from_task_id}→${d.to_task_id}`))
  const taskNameMap = new Map(currTasks.map(t => [t.id as string, t.name as string]))
  // fallback to prev names for deleted tasks
  for (const t of prevTasks) if (!taskNameMap.has(t.id as string)) taskNameMap.set(t.id as string, t.name as string)

  for (const d of currDeps) {
    const key = `${d.from_task_id}→${d.to_task_id}`
    if (!prevDepSet.has(key)) {
      const from = taskNameMap.get(d.from_task_id as string) ?? '未知'
      const to   = taskNameMap.get(d.to_task_id   as string) ?? '未知'
      changes.push({
        change_type: 'dep_add',
        task_name: null, field_name: null, old_value: null, new_value: null,
        description: `新增依赖: 「${from}」→「${to}」`,
      })
    }
  }
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
  for (const d of prevDeps) {
    const key = `${d.from_task_id}→${d.to_task_id}`
    if (!currDepSet.has(key)) {
      const from = taskNameMap.get(d.from_task_id as string) ?? '未知'
      const to   = taskNameMap.get(d.to_task_id   as string) ?? '未知'
      changes.push({
        change_type: 'dep_delete',
        task_name: null, field_name: null, old_value: null, new_value: null,
        description: `删除依赖: 「${from}」→「${to}」`,
      })
    }
  }

  return changes
}

// GET /api/versions/[projectId] — list versions with change logs
export async function GET(req: NextRequest, { params }: { params: Promise<{ projectId: string }> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId } = await params
  const client = await pool.connect()
  try {
    const proj = await client.query(
      'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
      [projectId, auth.value.userId],
    )
    if (!proj.rows.length)
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })

    const versRes = await client.query(
      `SELECT v.id, v.project_id, v.version_number, v.created_at,
              u.username AS created_by_name,
              jsonb_array_length(v.snapshot->'tasks') AS task_count
       FROM project_versions v
       LEFT JOIN users u ON u.id = v.created_by
       WHERE v.project_id = $1
       ORDER BY v.version_number DESC`,
      [projectId],
    )

    // Attach change logs to each version
    const versionIds = versRes.rows.map(v => v.id)
    let logsMap: Record<string, unknown[]> = {}
    if (versionIds.length > 0) {
      const logsRes = await client.query(
        `SELECT * FROM change_logs WHERE version_id = ANY($1) ORDER BY created_at ASC`,
        [versionIds],
      )
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
      logsMap = logsRes.rows.reduce((acc, l) => {
        if (!acc[l.version_id]) acc[l.version_id] = []
        acc[l.version_id].push(l)
        return acc
      }, {} as Record<string, unknown[]>)
    }

    const value = versRes.rows.map(v => ({ ...v, changes: logsMap[v.id] ?? [] }))
    return NextResponse.json({ ok: true, value })
  } finally {
    client.release()
  }
}

// POST /api/versions/[projectId] — auto-save new version with diff
export async function POST(req: NextRequest, { params }: { params: Promise<{ projectId: string }> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId } = await params

  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    const proj = await client.query(
      'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
      [projectId, auth.value.userId],
    )
    if (!proj.rows.length) {
      await client.query('ROLLBACK')
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })
    }

    // Current state
    const [tasksRes, depsRes] = await Promise.all([
      client.query('SELECT * FROM tasks WHERE project_id = $1 ORDER BY order_index', [projectId]),
      client.query('SELECT * FROM dependencies WHERE project_id = $1', [projectId]),
    ])
    const snapshot = { tasks: tasksRes.rows, dependencies: depsRes.rows }

    // Previous version snapshot for diffing
    const prevRes = await client.query(
      `SELECT snapshot FROM project_versions WHERE project_id = $1 ORDER BY version_number DESC LIMIT 1`,
      [projectId],
    )
    const prevSnap = prevRes.rows[0]?.snapshot as { tasks: TaskRow[]; dependencies: DepRow[] } | undefined
    const changes = prevSnap
      ? diffSnapshots(prevSnap.tasks, prevSnap.dependencies, tasksRes.rows, depsRes.rows)
      : tasksRes.rows.map(t => ({
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
            change_type: 'task_add' as const,
            task_name:   t.name as string,
            field_name:  null, old_value: null, new_value: null,
            description: `新增任务「${t.name}」`,
          }))

      // Next version number
      const maxRes = await client.query(
        'SELECT COALESCE(MAX(version_number), 0) AS mx FROM project_versions WHERE project_id = $1',
        [projectId],
      )
      const nextVersion = (maxRes.rows[0].mx as number) + 1

      const userRes = await client.query('SELECT username FROM users WHERE id = $1', [auth.value.userId])
      const username = userRes.rows[0]?.username ?? null

      const ins = await client.query(
        `INSERT INTO project_versions (project_id, version_number, snapshot, created_by)
         VALUES ($1, $2, $3, $4)
         RETURNING id, project_id, version_number, created_at`,
        [projectId, nextVersion, JSON.stringify(snapshot), auth.value.userId],
      )
      const versionId = ins.rows[0].id

      // Insert change logs
      for (const c of changes) {
        await client.query(
          `INSERT INTO change_logs (project_id, version_id, change_type, task_name, field_name, old_value, new_value, descript
           VALUES ($1,$2,$3,$4,$5,$6,$7,$8)`,
          [projectId, versionId, c.change_type, c.task_name, c.field_name, c.old_value, c.new_value, c.description],
        )
      }

      await client.query('COMMIT')
      return NextResponse.json({
        ok: true,
        value: { ...ins.rows[0], task_count: tasksRes.rows.length, created_by_name: username, changes },
      })
    } catch (err) {
      await client.query('ROLLBACK')
      console.error('Save version failed', err)
      return NextResponse.json({ ok: false, error: 'Save failed' }, { status: 500 })
    } finally {
      client.release()
    }
  }

```

// app/dashboard/page.tsx

```
'use client'

```

```
import { useEffect, useState } from 'react'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

```
import { useRouter } from 'next/navigation'
import { useAppDispatch, useAppSelector } from '@/store/hooks'
import { setProjects, addProject } from '@/store/slices/projectSlice'
import ProjectCard from '@/components/ProjectCard'
import { logout } from '@/store/slices/authSlice'

export default function DashboardPage() {
  const dispatch = useAppDispatch()
  const router = useRouter()
  const { user } = useAppSelector(s => s.auth)
  const { projects } = useAppSelector(s => s.project)
  const [creating, setCreating] = useState(false)
  const [newProjectName, setNewProjectName] = useState('')
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    if (!user) {
      router.push('/login')
      return
    }
    fetch('/api/projects')
      .then(r => r.json())
      .then(data => {
        if (data.ok) dispatch(setProjects(data.value))
        setLoading(false)
      })
  }, [user, dispatch, router])

  const handleCreate = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!newProjectName.trim()) return

    const res = await fetch('/api/projects', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        name: newProjectName,
        start_date: new Date().toISOString().split('T')[0],
      }),
    })
    const data = await res.json()
    if (data.ok) {
      dispatch(addProject(data.value))
      setNewProjectName('')
      setCreating(false)
    }
  }

  const handleLogout = async () => {
```

```
1200      await fetch('/api/auth/logout', { method: 'POST' }).catch(() => null)
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

```
      dispatch(logout())
      router.push('/login')
    }

    if (!user) return null

    return (
      <div className="min-h-screen bg-gray-50">
        <header className="bg-white border-b border-gray-200 px-6 py-4 flex items-center justify-between">
          <h1 className="text-xl font-bold text-gray-900">甘特图管理</h1>
          <div className="flex items-center gap-4">
            <span className="text-sm text-gray-600">你好，{user.username}</span>
            <button onClick={handleLogout} className="text-sm text-gray-500 hover:text-gray-700">
              退出登录
            </button>
          </div>
        </header>
        <main className="max-w-4xl mx-auto px-6 py-8">
          <div className="flex items-center justify-between mb-6">
            <h2 className="text-lg font-semibold text-gray-800">我的项目</h2>
            <button
              onClick={() => setCreating(true)}
              className="px-4 py-2 bg-blue-600 text-white text-sm rounded hover:bg-blue-700"
            >
              + 新建项目
            </button>
          </div>

          {creating && (
            <form onSubmit={handleCreate} className="mb-6 flex gap-2">
              <input
                autoFocus
                type="text"
                placeholder="项目名称"
                value={newProjectName}
                onChange={e => setNewProjectName(e.target.value)}
                className="flex-1 border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-
              />
              <button
                type="submit"
                className="px-4 py-2 bg-blue-600 text-white text-sm rounded hover:bg-blue-700"
              >
                创建
              </button>
              <button
                type="button"
                onClick={() => setCreating(false)}
                className="px-4 py-2 bg-gray-200 text-gray-700 text-sm rounded hover:bg-gray-300"
              >
```

1250 取消

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

```
              </button>
            </form>
          )}

          {loading ? (
            <div className="text-center text-gray-500 py-12">加载中...</div>
          ) : projects.length === 0 ? (
            <div className="text-center text-gray-500 py-12">
              暂无项目，点击「新建项目」开始
            </div>
          ) : (
            <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
              {projects.map(p => (
                <ProjectCard key={p.id} project={p} />
              ))}
            </div>
          )}
        </main>
      </div>
    )
  }

```

// app/globals.css

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}

```

// app/layout.tsx

```
import type { Metadata } from 'next'
import './globals.css'
import { StoreProvider } from '@/store/provider'

export const metadata: Metadata = {
  title: '跟踪甘特图',
```

```
  description: '项目进度甘特图管理工具',
}

export default function RootLayout({
  children,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

```
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <head>
        {/* Bryntum Gantt 7.2.1 library loaded before page scripts */}
        {/* eslint-disable-next-line @next/next/no-sync-scripts */}
        <script src="/lib/gantt/gantt.umd.js" />
      </head>
      <body>
        <StoreProvider>{children}</StoreProvider>
      </body>
    </html>
  )
}

```

// app/login/page.tsx

```
import Link from 'next/link'
import LoginForm from '@/components/auth/LoginForm'

export default function LoginPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <div className="bg-white rounded-lg shadow p-8 w-full max-w-md">
        <h1 className="text-2xl font-bold text-gray-900 mb-6">登录</h1>
        <LoginForm />
        <p className="mt-4 text-center text-sm text-gray-600">
          没有账号？{' '}
          <Link href="/register" className="text-blue-600 hover:underline">
            立即注册
          </Link>
        </p>
      </div>
    </div>
  )
}

```

// app/page.tsx

```
import { redirect } from 'next/navigation'

```

```
export default function Home() {
  redirect('/login')
}

```

// app/projects/[id]/page.tsx

```
'use client'

import { useEffect, useRef, useState } from 'react'
import { useParams, useRouter } from 'next/navigation'
import dynamic from 'next/dynamic'
import { useAppDispatch, useAppSelector } from '@/store/hooks'
import { setTasks, clearTasks } from '@/store/slices/tasksSlice'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

```
import { setCurrentProject } from '@/store/slices/projectSlice'
import GanttToolbar from '@/components/GanttChart/GanttToolbar'
import VersionPanel from '@/components/GanttChart/VersionPanel'
import Link from 'next/link'

const GanttChart = dynamic(() => import('@/components/GanttChart/GanttChart'), {
  ssr: false,
  loading: () => (
    <div className="flex items-center justify-center h-64 text-gray-500">
      Loading Gantt Chart...
    </div>
  ),
})

const COL_W_MIN  = 14
const COL_W_MAX  = 56
const COL_W_STEP = 7

export default function ProjectPage() {
  const params    = useParams()
  const projectId = params.id as string
  const dispatch  = useAppDispatch()
  const router    = useRouter()
  const { user }           = useAppSelector(s => s.auth)
  const { currentProject } = useAppSelector(s => s.project)
  const loadedRef = useRef(false)

  // ── Gantt UI state ──────────────────────────────────────────────
  const [colW,              setColW]              = useState(28)
  const [searchQuery,       setSearchQuery]       = useState('')
  const [expandAllSignal,   setExpandAllSignal]   = useState(0)
  const [collapseAllSignal, setCollapseAllSignal] = useState(0)
  const [focusSignal,       setFocusSignal]       = useState(0)
  const [showVersions,      setShowVersions]      = useState(false)

  useEffect(() => {
    if (!user) { router.push('/login'); return }
```

```
    if (loadedRef.current) return
    loadedRef.current = true

    fetch(`/api/projects/${projectId}`)
      .then(r => r.json())
      .then(data => { if (data.ok) dispatch(setCurrentProject(data.value)); else router.push('/dashboard') })

    fetch(`/api/tasks/${projectId}`)
      .then(r => r.json())
      .then(data => { if (data.ok) dispatch(setTasks(data.value)) })

    return () => {
      dispatch(clearTasks())
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

```
      dispatch(setCurrentProject(null))
    }
  }, [user, projectId, dispatch, router])

  if (!user) return null

  return (
    <div className="flex flex-col h-screen bg-white">
      <header className="bg-white border-b border-gray-200 px-6 py-3 flex items-center gap-4">
        <Link href="/dashboard" className="text-sm text-gray-500 hover:text-gray-700">
          ← 返回项目列表
        </Link>
        <h1 className="text-lg font-semibold text-gray-900">
          {currentProject?.name ?? '加载中...'}
        </h1>
      </header>

      <GanttToolbar
        projectId={projectId}
        colW={colW}
        onZoomIn={() => setColW(w => Math.min(COL_W_MAX, w + COL_W_STEP))}
        onZoomOut={() => setColW(w => Math.max(COL_W_MIN, w - COL_W_STEP))}
        onExpandAll={() => setExpandAllSignal(n => n + 1)}
        onCollapseAll={() => setCollapseAllSignal(n => n + 1)}
        onFocusTask={() => setFocusSignal(n => n + 1)}
        searchQuery={searchQuery}
        onSearchChange={setSearchQuery}
        onShowVersions={() => setShowVersions(true)}
      />

      {showVersions && (
        <VersionPanel projectId={projectId} onClose={() => setShowVersions(false)} />
      )}

      <div className="flex-1 overflow-hidden">
        <GanttChart
          projectId={projectId}
```

```
            statusDate={currentProject?.status_date}
            colW={colW}
            searchQuery={searchQuery}
            expandAllSignal={expandAllSignal}
            collapseAllSignal={collapseAllSignal}
            focusSignal={focusSignal}
          />
        </div>
      </div>
    )
  }

```

// app/register/page.tsx

```
import Link from 'next/link'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

```
import RegisterForm from '@/components/auth/RegisterForm'

export default function RegisterPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <div className="bg-white rounded-lg shadow p-8 w-full max-w-md">
        <h1 className="text-2xl font-bold text-gray-900 mb-6">创建账号</h1>
        <RegisterForm />
        <p className="mt-4 text-center text-sm text-gray-600">
          已有账号？{' '}
          <Link href="/login" className="text-blue-600 hover:underline">
            返回登录
          </Link>
        </p>
      </div>
    </div>
  )
}

```

// components/GanttChart/EditTaskModal.tsx

```
'use client'

import React, { useState, useEffect, useCallback } from 'react'
import { useAppDispatch, useAppSelector } from '@/store/hooks'
import { updateTasks, saveSnapshot } from '@/store/slices/tasksSlice'
import type { TaskLifecycleEvent } from '@/types'

interface Props {
  taskId: string
  projectId: string
  onClose: () => void
}
```

```

const EVENT_ICONS: Record<string, { icon: string; color: string }> = {
  created: { icon: '＋', color: 'text-green-600 bg-green-50' },
  updated: { icon: '✎',  color: 'text-blue-600 bg-blue-50'  },
  moved:   { icon: '⇅',  color: 'text-amber-600 bg-amber-50'},
  deleted: { icon: '－', color: 'text-red-500 bg-red-50'    },
}

export default function EditTaskModal({ taskId, projectId, onClose }: Props) {
  const dispatch = useAppDispatch()
  const task = useAppSelector(s => s.tasks.tasks.find(t => t.id === taskId))

  const [tab, setTab] = useState<'edit' | 'history'>('edit')

  const [name,        setName]        = useState('')
  const [assignee,    setAssignee]    = useState('')
  const [startDate,   setStartDate]   = useState('')
  const [endDate,     setEndDate]     = useState('')
  const [percentDone, setPercentDone] = useState(0)
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
  projectId,
  colW, onZoomIn, onZoomOut,
  onExpandAll, onCollapseAll,
  onFocusTask,
  searchQuery, onSearchChange,
  onShowVersions,
}: GanttToolbarProps) {
  const dispatch = useAppDispatch()
  const { selectedIds, clipboard, tasks, undoStack, redoStack } = useAppSelector(s => s.tasks)
  const currentProject = useAppSelector(s => s.project.currentProject)

  const canUndo = undoStack.length > 0
  const canRedo = redoStack.length > 0
  const hasSelection = selectedIds.length > 0

  const [editModalOpen, setEditModalOpen] = useState(false)
  const [saving, setSaving]   = useState(false)
  const [saved,  setSaved]    = useState(false)

  // ── Save version (auto, no label) ──────────────────────────────
  const handleSave = useCallback(async () => {
    if (saving) return
    setSaving(true)
    const res = await fetch(`/api/versions/${projectId}`, { method: 'POST' })
    const data = await res.json()
```

```
      setSaving(false)
      if (data.ok) {
        setSaved(true)
        setTimeout(() => setSaved(false), 2000)
      } else {
        alert('保存失败，请重试')
      }
    }, [projectId, saving])

    // ── Navigate prev/next task ──────────────────────────────────────────
    const flatOrder = getFlatOrder(tasks)
    const selectedIdx = hasSelection
      ? flatOrder.findIndex(t => t.id === selectedIds[0])
      : -1

    const handlePrev = useCallback(() => {
      if (selectedIdx <= 0) return
      dispatch(setSelectedIds([flatOrder[selectedIdx - 1].id]))
    }, [dispatch, flatOrder, selectedIdx])

    const handleNext = useCallback(() => {
      if (selectedIdx < 0 || selectedIdx >= flatOrder.length - 1) return
      dispatch(setSelectedIds([flatOrder[selectedIdx + 1].id]))
    }, [dispatch, flatOrder, selectedIdx])

```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
  // ── Create task ──────────────────────────────────────────────
  const handleAddTask = useCallback(async () => {
    // defaultStart: later of earliest task start and status date
    const sod = (d: Date) => { const r = new Date(d); r.setHours(0, 0, 0, 0); return r }
    const starts = tasks.filter(t => t.start_date).map(t => sod(new Date(t.start_date!)))
    let startD = starts.length > 0
      ? new Date(Math.min(...starts.map(d => d.getTime())))
      : sod(new Date())
    if (currentProject?.status_date) {
      const sd = sod(new Date(currentProject.status_date))
      if (sd > startD) startD = sd
    }
    const startStr = startD.toISOString().split('T')[0]
    const endD = new Date(startD); endD.setDate(endD.getDate() + 1)
    const endStr = endD.toISOString().split('T')[0]

    const rootTasks = tasks.filter(t => t.parent_id === null)
    const nextIndex = rootTasks.length > 0
      ? Math.max(...rootTasks.map(t => t.order_index)) + 1 : 0
    dispatch(saveSnapshot())
    const res = await fetch(`/api/tasks/${projectId}`, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        parent_id: null, name: 'New Task',
        start_date: startStr, end_date: endStr,
```

```
        duration: 1, duration_unit: 'day',
        percent_done: 0, is_milestone: false, note: null, order_index: nextIndex,
      }),
    })
    const data = await res.json()
    if (data.ok && data.value?.length > 0) {
      dispatch(addTasks(data.value))
      dispatch(setSelectedIds([data.value[0].id]))
      setEditModalOpen(true)
    }
  }, [dispatch, projectId, tasks, currentProject])

  // ── Delete ──────────────────────────────────────────────────────────
  const handleDeleteTasks = useCallback(async () => {
    if (!hasSelection) return
    if (!confirm(`确定删除 ${selectedIds.length} 个任务？`)) return
    dispatch(saveSnapshot())
    const res = await fetch(`/api/tasks/${projectId}`, {
      method: 'DELETE', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ ids: selectedIds }),
    })
    const data = await res.json()
    if (data.ok) dispatch(deleteTasks(data.value.deleted))
  }, [dispatch, projectId, selectedIds, hasSelection])

```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
  // ── Promote ──────────────────────────────────────────────────────
  const handlePromote = useCallback(async () => {
    if (!hasSelection) return
    const selectedSet = new Set(selectedIds)
    const toPromote = tasks.filter(t =>
      selectedSet.has(t.id) && t.parent_id !== null && !selectedSet.has(t.parent_id ?? '')
    )
    if (toPromote.length === 0) return
    const groups = new Map<string, Task[]>()
    toPromote.forEach(t => {
      if (!groups.has(t.parent_id!)) groups.set(t.parent_id!, [])
      groups.get(t.parent_id!)!.push(t)
    })
    const allUpdates: Array<{ id: string; parent_id: string | null; order_index: number }> = []
    const alreadyShifted = new Set<string>()
    for (const [parentId, group] of groups) {
      const parent = tasks.find(t => t.id === parentId)!
      const grandparentId = parent.parent_id
      const sortedGroup = [...group].sort((a, b) => a.order_index - b.order_index)
      tasks.filter(t =>
        t.parent_id === grandparentId && t.order_index > parent.order_index &&
        !selectedSet.has(t.id) && !alreadyShifted.has(t.id)
      ).forEach(t => {
        alreadyShifted.add(t.id)
        allUpdates.push({ id: t.id, parent_id: t.parent_id, order_index: t.order_index + sortedGroup.length })
```

```
      })
      sortedGroup.forEach((task, i) => {
        allUpdates.push({ id: task.id, parent_id: grandparentId, order_index: parent.order_index + 1 + i })
      })
    }
    dispatch(saveSnapshot())
    dispatch(updateTasks(allUpdates.map(u => ({ ...tasks.find(t => t.id === u.id)!, ...u }))))
    await fetch(`/api/tasks/${projectId}`, {
      method: 'PUT', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(allUpdates),
    })
  }, [dispatch, projectId, tasks, selectedIds, hasSelection])

  // ── Demote ──────────────────────────────────────────────────────────
  const handleDemote = useCallback(async () => {
    if (!hasSelection) return
    const selectedSet = new Set(selectedIds)
    const flatOrder = getFlatOrder(tasks)
    const toDemote = flatOrder.filter(t => selectedSet.has(t.id) && !selectedSet.has(t.parent_id ?? ''))
    if (toDemote.length === 0) return
    const firstTask = toDemote[0]
    const anchor = tasks
      .filter(t => t.parent_id === firstTask.parent_id && t.order_index < firstTask.order_index && !selectedSet.has(t.id))
      .sort((a, b) => b.order_index - a.order_index)[0]
    if (!anchor) return
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
    const existingChildren = tasks.filter(t => t.parent_id === anchor.id)
    const startOrder = existingChildren.length > 0
      ? Math.max(...existingChildren.map(t => t.order_index)) + 1 : 0
    const updates = toDemote.map((task, i) => ({ id: task.id, parent_id: anchor.id, order_index: startOrder + i }))
    dispatch(saveSnapshot())
    dispatch(updateTasks(updates.map(u => ({ ...tasks.find(t => t.id === u.id)!, ...u }))))
    await fetch(`/api/tasks/${projectId}`, {
      method: 'PUT', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(updates),
    })
  }, [dispatch, projectId, tasks, selectedIds, hasSelection])

  // ── Copy / Paste ──────────────────────────────────────────────────────
  const handleCopy = useCallback(() => {
    if (!hasSelection) return
    dispatch(copyTasks(selectedIds))
  }, [dispatch, selectedIds, hasSelection])

  const handlePaste = useCallback(async () => {
    if (clipboard.length === 0) return
    dispatch(saveSnapshot())
    const pastedTasks = clipboard.map(t => ({
      ...t, name: `${t.name} (copy)`, id: undefined, created_at: undefined, updated_at: undefined,
    }))
    const res = await fetch(`/api/tasks/${projectId}`, {
```

```
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(pastedTasks),
    })
    const data = await res.json()
    if (data.ok) dispatch(addTasks(data.value))
  }, [dispatch, projectId, clipboard])

  // ── Status date ──────────────────────────────────────────────────────
  const handleStatusDateChange = useCallback(async (e: React.ChangeEvent<HTMLInputElement>) => {
    const statusDate = e.target.value || null
    dispatch(setStatusDate({ projectId, statusDate }))
    if (statusDate && tasks.length > 0) {
      const sd = new Date(statusDate)
      const updated = tasks.filter(t => t.start_date && t.end_date).map(t => ({ ...t, percent_done: calcPercent(t, sd) }))
      dispatch(updateTasks(updated))
      await fetch(`/api/tasks/${projectId}`, {
        method: 'PUT', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(updated.map(t => ({ id: t.id, percent_done: t.percent_done }))),
      })
    }
    await fetch(`/api/projects/${projectId}`, {
      method: 'PUT', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status_date: statusDate }),
    })
  }, [dispatch, projectId, tasks])
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```

  // ── Keyboard shortcuts ──────────────────────────────────────────
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if (document.activeElement?.tagName === 'INPUT' || document.activeElement?.tagName === 'TEXTAREA') return
      if ((e.ctrlKey || e.metaKey) && e.key === 'z') { e.preventDefault(); dispatch(undo()) }
      if ((e.ctrlKey || e.metaKey) && (e.key === 'y' || (e.shiftKey && e.key === 'z'))) { e.preventDefault(); dispatch(redo(
      if ((e.ctrlKey || e.metaKey) && e.key === 'c') handleCopy()
      if ((e.ctrlKey || e.metaKey) && e.key === 'v') handlePaste()
    }
    window.addEventListener('keydown', handler)
    return () => window.removeEventListener('keydown', handler)
  }, [dispatch, handleCopy, handlePaste])

  const sep = <div className="w-px h-6 bg-gray-200 mx-1" />

  return (
    <>
      <div className="flex items-center gap-1.5 px-3 py-2 bg-white border-b border-gray-200 flex-wrap">

        {/* Group 1: Create + Edit */}
        <LabelIc icon={<IcoPlus />}   label="CREATE" variant="green" onClick={handleAddTask} />
        <LabelIc icon={<IcoPencil />} label="EDIT"   variant="blue"
                 disabled={selectedIds.length !== 1}
                 onClick={() => selectedIds.length === 1 && setEditModalOpen(true)} />
```

```
        {sep}

        {/* Group 2: Undo / Redo */}
        <Ic title="撤销 (Ctrl+Z)" disabled={!canUndo} onClick={() => dispatch(undo())}><IcoUndo /></Ic>
        <Ic title="重做 (Ctrl+Y)" disabled={!canRedo} onClick={() => dispatch(redo())}><IcoRedo /></Ic>

        {sep}

        {/* Group 3: Expand / Collapse all */}
        <Ic title="全部展开" onClick={onExpandAll}><IcoExpand /></Ic>
        <Ic title="全部折叠" onClick={onCollapseAll}><IcoCollapse /></Ic>

        {sep}

        {/* Group 4: Zoom + Focus + Indent/Outdent */}
        <Ic title={`放大 (当前: ${colW}px)`} disabled={colW >= 56} onClick={onZoomIn}><IcoZoomIn /></Ic>
        <Ic title={`缩小 (当前: ${colW}px)`} disabled={colW <= 14} onClick={onZoomOut}><IcoZoomOut /></Ic>
        <Ic title="聚焦到选中任务" disabled={!hasSelection} onClick={onFocusTask}><IcoFocus /></Ic>
        <Ic title="升级 (Outdent)" disabled={!hasSelection} onClick={handlePromote}><IcoPrev /></Ic>
        <Ic title="降级 (Indent)"  disabled={!hasSelection} onClick={handleDemote}><IcoNext /></Ic>

        {sep}

        {/* Search */}
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
          <div className="flex items-center gap-1 border border-gray-300 rounded px-2 h-8 bg-white">
            <IcoSearch />
            <input
              type="text"
              placeholder="搜索任务..."
              value={searchQuery}
              onChange={e => onSearchChange(e.target.value)}
              className="w-36 text-[13px] outline-none placeholder:text-gray-400"
            />
            {searchQuery && (
              <button onClick={() => onSearchChange('')} className="text-gray-400 hover:text-gray-600 text-sm">×</button>
            )}
          </div>

          {sep}

          {/* Secondary: Copy / Paste / Delete */}
          <Ic title="复制 (Ctrl+C)" disabled={!hasSelection} onClick={handleCopy}>
            <svg viewBox="0 0 16 16" width="14" height="14" fill="none" stroke="currentColor" strokeWidth="1.8">
              <rect x="5" y="5" width="8" height="9" rx="1"/><path d="M3 11V3h8" strokeLinecap="round"/>
            </svg>
          </Ic>
          <Ic title="粘贴 (Ctrl+V)" disabled={clipboard.length === 0} onClick={handlePaste}>
            <svg viewBox="0 0 16 16" width="14" height="14" fill="none" stroke="currentColor" strokeWidth="1.8">
              <path d="M5 3h6v2H5V3z"/><rect x="3" y="4" width="10" height="10" rx="1"/>
```

```
            </svg>
          </Ic>
          <Ic title="删除选中任务" disabled={!hasSelection} onClick={handleDeleteTasks}>
            <svg viewBox="0 0 16 16" width="14" height="14" fill="none" stroke="currentColor" strokeWidth="1.8">
              <path d="M3 5h10M6 5V3h4v2M7 8v4M9 8v4" strokeLinecap="round"/>
              <path d="M4 5l1 9h6l1-9H4z"/>
            </svg>
          </Ic>

          {sep}

          {/* Status date */}
          <div className="flex items-center gap-1.5">
            <span className="text-xs text-gray-500 whitespace-nowrap">状态日期</span>
            <input
              type="date"
              value={currentProject?.status_date?.split('T')[0] ?? ''}
              onChange={handleStatusDateChange}
              className="border border-gray-300 rounded px-2 h-8 text-[13px] focus:outline-none focus:border-blue-400"
            />
          </div>

          {sep}

          {/* Save version */}
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
        <button
          disabled={saving}
          onClick={handleSave}
          className={`inline-flex items-center gap-1.5 px-2.5 h-8 rounded border text-[13px] font-medium transition-colors
            ${saved
              ? 'border-green-500 text-green-600 bg-green-50'
              : 'border-green-400 text-green-600 hover:bg-green-50 bg-white cursor-pointer'}
            disabled:opacity-50 disabled:cursor-not-allowed`}>
          {saved ? (
            <>
              <svg viewBox="0 0 16 16" width="13" height="13" fill="none" stroke="currentColor" strokeWidth="2.5">
                <path d="M3 8l4 4 6-6" strokeLinecap="round" strokeLinejoin="round"/>
              </svg>
              已保存
            </>
          ) : (
            <>
              <svg viewBox="0 0 16 16" width="13" height="13" fill="none" stroke="currentColor" strokeWidth="2">
                <path d="M2 12V4l3-2h6l3 2v8a1 1 0 01-1 1H3a1 1 0 01-1-1z"/>
                <path d="M5 14V9h6v5M10 2v4H5V2" strokeLinecap="round" strokeLinejoin="round"/>
              </svg>
              {saving ? '保存中...' : '保存版本'}
            </>
          )}
        </button>
```

```
          {/* Version history */}
          <button
            onClick={onShowVersions}
            title="版本历史"
            className="inline-flex items-center gap-1.5 px-2.5 h-8 rounded border text-[13px] font-medium
                       border-gray-300 text-gray-600 hover:bg-gray-50 bg-white cursor-pointer">
            <svg viewBox="0 0 16 16" width="13" height="13" fill="none" stroke="currentColor" strokeWidth="1.8">
              <circle cx="8" cy="8" r="6"/>
              <path d="M8 5v3.5l2 1.5" strokeLinecap="round" strokeLinejoin="round"/>
            </svg>
            历史
          </button>
        </div>

        {/* Edit Task Modal */}
        {editModalOpen && selectedIds.length === 1 && (
          <EditTaskModal
            taskId={selectedIds[0]}
            projectId={projectId}
            onClose={() => setEditModalOpen(false)}
          />
        )}
      </>
    )
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

```
}

```

// components/GanttChart/VersionPanel.tsx

```
'use client'

import React, { useEffect, useState, useCallback } from 'react'
import { useAppDispatch } from '@/store/hooks'
import { setTasks, saveSnapshot } from '@/store/slices/tasksSlice'
import type { ProjectVersion, ChangeLog } from '@/types'

interface Props {
  projectId: string
  onClose: () => void
}

const CHANGE_ICONS: Record<string, string> = {
  task_add:     '＋',
  task_delete:  '－',
  task_update:  '✎',
  task_reorder: '⇅',
  dep_add:      '→',
  dep_delete:   '✕',
}
```

```
const CHANGE_COLORS: Record<string, string> = {
  task_add:     'text-green-600 bg-green-50',
  task_delete:  'text-red-500 bg-red-50',
  task_update:  'text-blue-600 bg-blue-50',
  task_reorder: 'text-amber-600 bg-amber-50',
  dep_add:      'text-purple-600 bg-purple-50',
  dep_delete:   'text-orange-500 bg-orange-50',
}

function ChangeItem({ c }: { c: ChangeLog }) {
  const icon  = CHANGE_ICONS[c.change_type]  ?? '·'
  const color = CHANGE_COLORS[c.change_type] ?? 'text-gray-500 bg-gray-100'
  return (
    <div className="flex items-start gap-2 py-1">
      <span className={`flex-none text-[10px] font-bold w-5 h-5 flex items-center justify-center rounded ${color}`}>
        {icon}
      </span>
      <span className="text-[12px] text-gray-600 leading-5">{c.description}</span>
    </div>
  )
}

export default function VersionPanel({ projectId, onClose }: Props) {
  const dispatch = useAppDispatch()
  const [versions,   setVersions]   = useState<ProjectVersion[]>([])
  const [loading,    setLoading]    = useState(true)
  const [restoring,  setRestoring]  = useState<string | null>(null)
  const [expandedId, setExpandedId] = useState<string | null>(null)
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

```

  const load = useCallback(() => {
    setLoading(true)
    fetch(`/api/versions/${projectId}`)
      .then(r => r.json())
      .then(d => { if (d.ok) setVersions(d.value) })
      .finally(() => setLoading(false))
  }, [projectId])

  useEffect(() => { load() }, [load])

  const handleRestore = useCallback(async (v: ProjectVersion) => {
    if (!confirm(`确定恢复到版本 v${v.version_number}? 当前未保存的改动将丢失。`)) return
    setRestoring(v.id)
    dispatch(saveSnapshot())
    const res  = await fetch(`/api/versions/${projectId}/${v.id}`, { method: 'POST' })
    const data = await res.json()
    if (data.ok) {
      dispatch(setTasks(data.value))
      onClose()
    } else {
      alert('恢复失败，请重试')
```

```
      }
      setRestoring(null)
    }, [dispatch, projectId, onClose])

    const fmtDate = (s: string) => {
      const d = new Date(s)
      return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2,'0')}-${String(d.getDate()).padStart(2,'0')} ${String(d.g
    }

    return (
      <div className="fixed inset-0 z-50 flex justify-end" onClick={onClose}>
        <div className="absolute inset-0 bg-black/20" />

        <div className="relative w-96 h-full bg-white shadow-xl flex flex-col"
             onClick={e => e.stopPropagation()}>

          {/* Header */}
          <div className="flex items-center justify-between px-4 py-3 border-b border-gray-200 flex-none">
            <h2 className="font-semibold text-gray-800 text-[15px]">版本历史</h2>
            <button onClick={onClose}
                    className="text-gray-400 hover:text-gray-600 text-xl leading-none w-7 h-7 flex items-center justify-center
              ×
            </button>
          </div>

          {/* List */}
          <div className="flex-1 overflow-y-auto">
            {loading ? (
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

```
            <div className="flex items-center justify-center h-32 text-gray-400 text-sm">加载中...</div>
          ) : versions.length === 0 ? (
            <div className="flex flex-col items-center justify-center h-48 text-gray-400 text-sm gap-2">
              <svg viewBox="0 0 24 24" width="32" height="32" fill="none" stroke="currentColor" strokeWidth="1.5">
                <path d="M12 8v4l3 3m6-3a9 9 0 11-18 0 9 9 0 0118 0z" strokeLinecap="round"/>
              </svg>
              暂无保存版本
            </div>
          ) : (
            <ul>
              {versions.map((v, idx) => {
                const isOpen = expandedId === v.id
                return (
                  <li key={v.id} className="border-b border-gray-100">
                    {/* Version header row */}
                    <div
                      className={`flex items-start gap-3 px-4 py-3 cursor-pointer transition-colors
                        ${isOpen ? 'bg-blue-50' : 'hover:bg-gray-50'}`}
                      onClick={() => setExpandedId(isOpen ? null : v.id)}>

                      {/* Timeline dot */}
                      <div className="flex flex-col items-center flex-none mt-1">
```

```
                        <div className={`w-2.5 h-2.5 rounded-full border-2
                          ${idx === 0 ? 'border-blue-500 bg-blue-500' : 'border-gray-300 bg-white'}`} />
                        {idx < versions.length - 1 && (
                          <div className="w-px flex-1 min-h-[20px] bg-gray-200 mt-1" />
                        )}
                      </div>

                      <div className="flex-1 min-w-0">
                        <div className="flex items-center justify-between gap-2">
                          <div className="flex items-center gap-2">
                            <span className={`text-[11px] font-bold px-1.5 py-0.5 rounded
                              ${idx === 0 ? 'bg-blue-100 text-blue-700' : 'bg-gray-100 text-gray-500'}`}>
                              v{v.version_number}
                            </span>
                            {idx === 0 && (
                              <span className="text-[10px] text-blue-600 bg-blue-50 border border-blue-200 rounded px-1">当前
                            )}
                          </div>
                          <svg viewBox="0 0 16 16" width="12" height="12" fill="none" stroke="currentColor" strokeWidth="2"
                               className={`flex-none text-gray-400 transition-transform ${isOpen ? 'rotate-180' : ''}`}>
                            <path d="M4 6l4 4 4-4" strokeLinecap="round" strokeLinejoin="round"/>
                          </svg>
                        </div>

                        <div className="mt-0.5 text-[12px] text-gray-500">{fmtDate(v.created_at)}</div>

                        <div className="mt-1 flex items-center gap-3 text-[11px] text-gray-400">
                          <span>{v.task_count} 个任务</span>
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

```
                          {v.created_by_name && <span>by {v.created_by_name}</span>}
                          {v.changes.length > 0 && (
                            <span className="text-blue-500">{v.changes.length} 项变更</span>
                          )}
                        </div>
                      </div>
                    </div>

                    {/* Expanded: change log + restore button */}
                    {isOpen && (
                      <div className="px-4 pb-3">
                        {v.changes.length === 0 ? (
                          <p className="text-[12px] text-gray-400 pl-5 py-1">暂无变更记录（初始版本）</p>
                        ) : (
                          <div className="pl-5 border-l-2 border-blue-100 ml-1 mb-2">
                            {v.changes.map(c => <ChangeItem key={c.id} c={c} />)}
                          </div>
                        )}
                        <button
                          disabled={!!restoring || idx === 0}
                          onClick={() => handleRestore(v)}
                          className="mt-1 w-full py-1.5 rounded text-[12px] font-medium border transition-colors
```

```
                              disabled:opacity-40 disabled:cursor-not-allowed
                              enabled:bg-blue-600 enabled:text-white enabled:border-blue-600 enabled:hover:bg-blue-700
                              bg-blue-600 text-white border-blue-600 hover:bg-blue-700">
                            {restoring === v.id ? '恢复中...' : idx === 0 ? '当前版本' : '恢复到此版本'}
                          </button>
                        </div>
                      )}
                    </li>
                  )
                })}
              </ul>
            )}
          </div>

          <div className="px-4 py-2 border-t border-gray-100 text-[11px] text-gray-400 text-center flex-none">
            共 {versions.length} 个版本
          </div>
        </div>
      </div>
    )
  }

```

// components/ProjectCard.tsx

```
'use client'

import Link from 'next/link'
import { useAppDispatch } from '@/store/hooks'
import { deleteProject } from '@/store/slices/projectSlice'
import type { Project } from '@/types'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

```

interface ProjectCardProps {
  project: Project
}

export default function ProjectCard({ project }: ProjectCardProps) {
  const dispatch = useAppDispatch()

  const handleDelete = async (e: React.MouseEvent) => {
    e.preventDefault()
    if (!confirm(`确定删除项目「${project.name}」？`)) return

    const res = await fetch(`/api/projects/${project.id}`, { method: 'DELETE' })
    const data = await res.json()
    if (data.ok) {
      dispatch(deleteProject(project.id))
    }
  }

  return (
```

```
      <div className="border border-gray-200 rounded-lg p-4 hover:shadow-md transition-shadow bg-white">
        <div className="flex items-start justify-between">
          <Link href={`/projects/${project.id}`} className="flex-1">
            <h3 className="font-semibold text-gray-900 hover:text-blue-600">{project.name}</h3>
            <div className="mt-1 text-sm text-gray-500 space-y-1">
              {project.start_date && (
                <p>开始：{new Date(project.start_date).toLocaleDateString('zh-CN')}</p>
              )}
              {project.status_date && (
                <p className="text-blue-600">
                  状态日期：{new Date(project.status_date).toLocaleDateString('zh-CN')}
                </p>
              )}
            </div>
          </Link>
          <button
            onClick={handleDelete}
            className="ml-2 text-gray-400 hover:text-red-600 text-sm"
            title="删除项目"
          >
            ×
          </button>
        </div>
      </div>
    )
  }

```

// components/auth/LoginForm.tsx

```
  'use client'

  import { useState } from 'react'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

```
import { useRouter } from 'next/navigation'
import { useAppDispatch } from '@/store/hooks'
import { setCredentials } from '@/store/slices/authSlice'

export default function LoginForm() {
  const dispatch = useAppDispatch()
  const router = useRouter()
  const [login, setLogin] = useState('')
  const [password, setPassword] = useState('')
  const [localError, setLocalError] = useState<string | null>(null)
  const [loading, setLocalLoading] = useState(false)

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setLocalError(null)
    setLocalLoading(true)

```

```
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ login, password }),
    })
    const data = await res.json()
    setLocalLoading(false)

    if (data.ok) {
      dispatch(setCredentials(data.value))
      router.push('/dashboard')
    } else {
      setLocalError(data.error)
    }
  }

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      {localError && (
        <div className="bg-red-50 text-red-700 p-3 rounded text-sm">{localError}</div>
      )}
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">用户名或邮箱</label>
        <input
          type="text"
          required
          value={login}
          onChange={e => setLogin(e.target.value)}
          className="w-full border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">密码</label>
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

```
          <input
            type="password"
            required
            value={password}
            onChange={e => setPassword(e.target.value)}
            className="w-full border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue
          />
        </div>
        <button
          type="submit"
          disabled={loading}
          className="w-full bg-blue-600 text-white py-2 rounded hover:bg-blue-700 disabled:opacity-50 text-sm font-medium"
        >
          {loading ? '登录中...' : '登录'}
        </button>
      </form>
    )
```

```
}

// components/auth/RegisterForm.tsx
'use client'

import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { useAppDispatch } from '@/store/hooks'
import { setCredentials } from '@/store/slices/authSlice'

export default function RegisterForm() {
  const dispatch = useAppDispatch()
  const router = useRouter()
  const [username, setUsername] = useState('')
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [localError, setLocalError] = useState<string | null>(null)
  const [loading, setLoading] = useState(false)

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setLocalError(null)
    setLoading(true)

    const res = await fetch('/api/auth/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, email, password }),
    })
    const data = await res.json()
    setLoading(false)

    if (data.ok) {
      dispatch(setCredentials(data.value))
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

// components/auth/RegisterForm.tsx

```
      router.push('/dashboard')
    } else {
      setLocalError(data.error)
    }
  }

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      {localError && (
        <div className="bg-red-50 text-red-700 p-3 rounded text-sm">{localError}</div>
      )}
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">用户名</label>
        <input
```

```
          type="text"
          required
          value={username}
          onChange={e => setUsername(e.target.value)}
          className="w-full border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">邮箱</label>
        <input
          type="email"
          required
          value={email}
          onChange={e => setEmail(e.target.value)}
          className="w-full border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">密码</label>
        <input
          type="password"
          required
          minLength={6}
          value={password}
          onChange={e => setPassword(e.target.value)}
          className="w-full border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue
        />
      </div>
      <button
        type="submit"
        disabled={loading}
        className="w-full bg-blue-600 text-white py-2 rounded hover:bg-blue-700 disabled:opacity-50 text-sm font-medium"
      >
        {loading ? '注册中...' : '注册'}
      </button>
    </form>
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

// components/auth/RegisterForm.tsx

```
  )
}

```

// lib/auth.ts

```
import jwt from 'jsonwebtoken'
import { type Result, success, failure } from './result'

const JWT_SECRET = process.env.JWT_SECRET ?? 'gantt-secret-key-change-in-production'
const JWT_EXPIRES_IN = '7d'

export interface JwtPayload {
  userId: string
```

```
  username: string
}

export function signToken(payload: JwtPayload): string {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN })
}

export function verifyToken(token: string): Result<JwtPayload> {
  try {
    const payload = jwt.verify(token, JWT_SECRET) as JwtPayload
    return success(payload)
  } catch {
    return failure('Invalid or expired token', 401)
  }
}

```

// lib/db.ts

```
import { Pool } from 'pg'

const globalForPg = globalThis as unknown as { pgPool: Pool | undefined }

export const pool =
  globalForPg.pgPool ??
  new Pool({
    host: 'localhost',
    port: 5432,
    database: 'gantt_db',
    user: 'postgres',
    password: '11111a',
  })

if (process.env.NODE_ENV !== 'production') {
  globalForPg.pgPool = pool
}

export default pool

```

// lib/middleware.ts

```
import { type NextRequest } from 'next/server'
import { verifyToken, type JwtPayload } from './auth'
import { type Result, failure } from './result'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

// components/auth/RegisterForm.tsx

// lib/auth.ts

// lib/db.ts

// lib/middleware.ts

```

export function getAuthUser(req: NextRequest): Result<JwtPayload> {
  const cookie = req.cookies.get('token')?.value
  const header = req.headers.get('authorization')?.replace('Bearer ', '')
  const token = cookie ?? header
```

```

  if (!token) {
    return failure('Unauthorized', 401)
  }

  return verifyToken(token)
}

```

// lib/result.ts

```
export type Success<T> = { ok: true; value: T }
export type Failure = { ok: false; error: string; code?: number }
export type Result<T> = Success<T> | Failure

export function success<T>(value: T): Success<T> {
  return { ok: true, value }
}

export function failure(error: string, code?: number): Failure {
  return { ok: false, error, code }
}

export function isSuccess<T>(result: Result<T>): result is Success<T> {
  return result.ok === true
}

export function isFailure<T>(result: Result<T>): result is Failure {
  return result.ok === false
}

```

// next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "./.next/dev/types/routes.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.

```

// next.config.ts

```
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  // Allow large static files (Bryntum library)
  experimental: {
    largePageDataBytes: 256 * 1024 * 1024,
  },
  // Empty turbopack config to suppress webpack conflict warning
  turbopack: {},
}
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

```

export default nextConfig

```

// postcss.config.mjs

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;

```

// scripts/check-users.ts

```
import { Pool } from 'pg'

const pool = new Pool({ host: 'localhost', port: 5432, database: 'gantt_db', user: 'postgres', password: '11111a' })

async function main() {
  const r = await pool.query('SELECT id, username, email, created_at FROM users')
  console.log('Users in DB:', r.rows.length)
  console.log(JSON.stringify(r.rows, null, 2))
  await pool.end()
}

main().catch(console.error)

```

// scripts/migrate.ts

```
import { Pool } from 'pg'

const pool = new Pool({
  host: 'localhost',
  port: 5432,
  user: 'postgres',
  password: '11111a',
})

async function migrate() {
  const client = await pool.connect()
  try {
    await client.query('CREATE DATABASE gantt_db')
    console.log('Created database gantt_db')
  } catch (err: unknown) {
    if (err && typeof err === 'object' && 'code' in err && err.code === '42P04') {
      console.log('Database gantt_db already exists')
    } else {
      throw err
    }
  } finally {
    client.release()
  }
  await pool.end()

  const dbPool = new Pool({
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

```
    host: 'localhost',
    port: 5432,
    database: 'gantt_db',
    user: 'postgres',
    password: '11111a',
  })

  const dbClient = await dbPool.connect()
  try {
    await dbClient.query(`
      CREATE TABLE IF NOT EXISTS users (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        username VARCHAR(50) UNIQUE NOT NULL,
        email VARCHAR(255) UNIQUE NOT NULL,
        password_hash VARCHAR(255) NOT NULL,
        created_at TIMESTAMPTZ DEFAULT NOW()
      );

      CREATE TABLE IF NOT EXISTS projects (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        user_id UUID REFERENCES users(id) ON DELETE CASCADE,
        name VARCHAR(255) NOT NULL,
        start_date DATE,
        end_date DATE,
        status_date DATE,
        created_at TIMESTAMPTZ DEFAULT NOW()
      );

      CREATE TABLE IF NOT EXISTS tasks (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
        parent_id UUID REFERENCES tasks(id) ON DELETE SET NULL,
        name VARCHAR(255) NOT NULL,
        assignee VARCHAR(100),
        start_date TIMESTAMPTZ,
        end_date TIMESTAMPTZ,
        duration NUMERIC,
        duration_unit VARCHAR(20) DEFAULT 'day',
        percent_done NUMERIC DEFAULT 0,
        is_milestone BOOLEAN DEFAULT false,
        note TEXT,
        order_index INTEGER DEFAULT 0,
        created_at TIMESTAMPTZ DEFAULT NOW(),
        updated_at TIMESTAMPTZ DEFAULT NOW()
      );

      CREATE TABLE IF NOT EXISTS dependencies (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
        from_task_id UUID REFERENCES tasks(id) ON DELETE CASCADE,
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

```
        to_task_id UUID REFERENCES tasks(id) ON DELETE CASCADE,
        type INTEGER DEFAULT 2,
        lag NUMERIC DEFAULT 0
      );
    `)
    // Add new columns to existing tables (idempotent)
    await dbClient.query(`
      ALTER TABLE tasks ADD COLUMN IF NOT EXISTS assignee VARCHAR(100);
      ALTER TABLE tasks ADD COLUMN IF NOT EXISTS task_code VARCHAR(20);
      ALTER TABLE tasks ADD COLUMN IF NOT EXISTS is_deleted BOOLEAN DEFAULT false NOT NULL;
      ALTER TABLE tasks ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ;
      CREATE UNIQUE INDEX IF NOT EXISTS idx_tasks_task_code ON tasks(project_id, task_code) WHERE task_code IS NOT NULL;
    `)

    // Task lifecycle table
    await dbClient.query(`
      CREATE TABLE IF NOT EXISTS task_lifecycle (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        task_id UUID NOT NULL,
        task_code VARCHAR(20) NOT NULL,
        project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
        event_type VARCHAR(30) NOT NULL,
        field_name VARCHAR(50),
        old_value TEXT,
        new_value TEXT,
        description TEXT NOT NULL,
        created_by UUID REFERENCES users(id) ON DELETE SET NULL,
        created_at TIMESTAMPTZ DEFAULT NOW()
      );
      CREATE INDEX IF NOT EXISTS idx_task_lifecycle_task ON task_lifecycle(task_id, created_at DESC);
      CREATE INDEX IF NOT EXISTS idx_task_lifecycle_project ON task_lifecycle(project_id, created_at DESC);
    `)
    // Version history table
    await dbClient.query(`
      CREATE TABLE IF NOT EXISTS project_versions (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
        version_number INTEGER NOT NULL,
        snapshot JSONB NOT NULL,
        created_by UUID REFERENCES users(id) ON DELETE SET NULL,
        created_at TIMESTAMPTZ DEFAULT NOW()
      );
      CREATE INDEX IF NOT EXISTS idx_project_versions_project
        ON project_versions(project_id, version_number DESC);

      CREATE TABLE IF NOT EXISTS change_logs (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
        version_id UUID NOT NULL REFERENCES project_versions(id) ON DELETE CASCADE,
        change_type VARCHAR(30) NOT NULL,
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

```
        task_name VARCHAR(255),
        field_name VARCHAR(50),
        old_value TEXT,
        new_value TEXT,
        description TEXT NOT NULL,
        created_at TIMESTAMPTZ DEFAULT NOW()
      );
      CREATE INDEX IF NOT EXISTS idx_change_logs_version
        ON change_logs(version_id);
    `)
    console.log('Schema migration complete')
  } finally {
    dbClient.release()
    await dbPool.end()
  }
}

migrate().catch(console.error)

```

// scripts/seed-admin.ts

```
import { Pool } from 'pg'
import bcrypt from 'bcryptjs'

const pool = new Pool({ host: 'localhost', port: 5432, database: 'gantt_db', user: 'postgres', password: '11111a' })

async function main() {
  const passwordHash = await bcrypt.hash('admin123', 10)
  const r = await pool.query(
    `INSERT INTO users (username, email, password_hash)
     VALUES ($1, $2, $3)
     ON CONFLICT (username) DO UPDATE SET password_hash = $3
     RETURNING id, username, email`,
    ['admin', 'admin@gantt.local', passwordHash]
  )
  console.log('Admin user ready:', r.rows[0])
  await pool.end()
}

main().catch(console.error)

```

// scripts/seed-demo.ts

```
/**
 * Seeds the "Launch SaaS Product" demo project matching the reference image.
 * Run: npx tsx scripts/seed-demo.ts
 */
import { Pool } from 'pg'

const pool = new Pool({
  host: 'localhost', port: 5432,
  database: 'gantt_db', user: 'postgres', password: '11111a',
})

```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

```
4507  async function main() {
4508    const client = await pool.connect()
4509    try {
4510      // Find the admin user
4511      const userRes = await client.query(`SELECT id FROM users WHERE username='admin' LIMIT 1`)
4512      if (userRes.rows.length === 0) {
4513        console.error('Admin user not found. Run: npx tsx scripts/seed-admin.ts first')
4514        return
4515      }
4516      const userId = userRes.rows[0].id
4517
4518      // ── Create project ──────────────────────────────────────────────────
4519      const projRes = await client.query(
4520        `INSERT INTO projects (user_id, name, start_date, end_date, status_date)
4521         VALUES ($1, 'Launch SaaS Product', '2026-01-12', '2026-02-09', '2026-01-22')
4522         RETURNING id`,
4523        [userId]
4524      )
4525      const pid = projRes.rows[0].id
4526      console.log('Created project:', pid)
4527
4528      // ── Helper ──────────────────────────────────────────────────────────
4529      let order = 0
4530      async function insertTask(
4531        name: string, parentId: string | null,
4532        start: string, end: string,
4533        percentDone: number, orderIdx?: number
4534      ): Promise<string> {
4535        const r = await client.query(
4536          `INSERT INTO tasks
4537             (project_id, parent_id, name, start_date, end_date, duration,
4538              duration_unit, percent_done, order_index)
4539           VALUES ($1,$2,$3,$4::date,$5::date,
4540             ($5::date - $4::date),
4541             'day', $6, $7)
4542           RETURNING id`,
4543          [pid, parentId, name, start, end, percentDone, orderIdx ?? order++]
4544        )
4545        return r.rows[0].id
4546      }
4547
4548      // ── Tasks (matching reference image) ────────────────────────────────
4549      const root    = await insertTask('Launch SaaS Product', null,
4550                        '2026-01-12', '2026-02-09', 0)
4551
4552      const server  = await insertTask('Setup web server',    root,
4553                        '2026-01-12', '2026-01-22', 0)
4554
4555      const apache  = await insertTask('Install Apache',      server,
4556                        '2026-01-12', '2026-01-17', 50)
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

```
    const fw      = await insertTask('Configure firewall',  server,
                        '2026-01-12', '2026-01-17', 50)
    const lb      = await insertTask('Setup load balancer', server,
                        '2026-01-12', '2026-01-17', 50)
    const ports   = await insertTask('Configure ports',     server,
                        '2026-01-12', '2026-01-14', 10)
    const tests   = await insertTask('Run tests',           server,
                        '2026-01-20', '2026-01-23', 0)

    const design  = await insertTask('Website Design',      root,
                        '2026-01-24', '2026-02-09', 0)

    const contact = await insertTask('Contact designers',            design,
                        '2026-01-24', '2026-02-01', 60)
    const short   = await insertTask('Create shortlist of three',    design,
                        '2026-02-01', '2026-02-05', 0)
    const select  = await insertTask('Select & review final design', design,
                        '2026-02-03', '2026-02-09', 0)

    // ── Dependencies ──────────────────────────────────────────────────────
    const deps = [
      [apache,  tests],
      [fw,      tests],
      [lb,      tests],
      [ports,   tests],
      [tests,   contact],
      [contact, short],
      [short,   select],
    ]
    for (const [from, to] of deps) {
      await client.query(
        `INSERT INTO dependencies (project_id, from_task_id, to_task_id, type)
         VALUES ($1, $2, $3, 2)`,
        [pid, from, to]
      )
    }

    console.log('Demo data seeded successfully!')
    console.log('Login: admin / admin123')
  } finally {
    client.release()
    await pool.end()
  }
}

main().catch(console.error)

```

// store/hooks.ts

```
import { useDispatch, useSelector, type TypedUseSelectorHook } from 'react-redux'
import type { RootState, AppDispatch } from './index'

```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

```
export const useAppDispatch = () => useDispatch<AppDispatch>()
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector

```

// store/index.ts

```
import { configureStore } from '@reduxjs/toolkit'
import authReducer from './slices/authSlice'
import projectReducer from './slices/projectSlice'
import tasksReducer from './slices/tasksSlice'

export const store = configureStore({
  reducer: {
    auth: authReducer,
    project: projectReducer,
    tasks: tasksReducer,
  },
})

export type RootState = ReturnType<typeof store.getState>
export type AppDispatch = typeof store.dispatch

```

// store/provider.tsx

```
'use client'

import { Provider } from 'react-redux'
import { store } from './index'

export function StoreProvider({ children }: { children: React.ReactNode }) {
  return <Provider store={store}>{children}</Provider>
}

```

// store/slices/authSlice.ts

```
import { createSlice, type PayloadAction } from '@reduxjs/toolkit'
import type { User } from '@/types'

interface AuthState {
  user: Omit<User, 'created_at'> | null
  token: string | null
  loading: boolean
  error: string | null
}

const initialState: AuthState = {
  user: null,
  token: null,
  loading: false,
  error: null,
}

const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    setCredentials(state, action: PayloadAction<{ user: Omit<User, 'created_at'>; token: string }>) {
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

// store/index.ts

// store/provider.tsx

// store/slices/authSlice.ts

```
      state.user = action.payload.user
      state.token = action.payload.token
      state.error = null
    },
    setLoading(state, action: PayloadAction<boolean>) {
      state.loading = action.payload
    },
    setError(state, action: PayloadAction<string | null>) {
      state.error = action.payload
    },
    logout(state) {
      state.user = null
      state.token = null
      state.error = null
    },
  },
})

export const { setCredentials, setLoading, setError, logout } = authSlice.actions
export default authSlice.reducer

```

// store/slices/projectSlice.ts

```
import { createSlice, type PayloadAction } from '@reduxjs/toolkit'
import type { Project } from '@/types'

interface ProjectState {
  projects: Project[]
  currentProject: Project | null
  loading: boolean
  error: string | null
}

const initialState: ProjectState = {
  projects: [],
  currentProject: null,
  loading: false,
  error: null,
}

const projectSlice = createSlice({
  name: 'project',
  initialState,
  reducers: {
    setProjects(state, action: PayloadAction<Project[]>) {
      state.projects = action.payload
    },
    addProject(state, action: PayloadAction<Project>) {
      state.projects.unshift(action.payload)
    },
    updateProject(state, action: PayloadAction<Project>) {
      const idx = state.projects.findIndex(p => p.id === action.payload.id)
```

```
// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

// components/auth/RegisterForm.tsx

// lib/auth.ts

// lib/db.ts

// lib/middleware.ts

// lib/result.ts

// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

// store/index.ts

// store/provider.tsx

// store/slices/authSlice.ts

// store/slices/projectSlice.ts

```
      if (idx !== -1) state.projects[idx] = action.payload
      if (state.currentProject?.id === action.payload.id) {
        state.currentProject = action.payload
      }
    },
    deleteProject(state, action: PayloadAction<string>) {
      state.projects = state.projects.filter(p => p.id !== action.payload)
      if (state.currentProject?.id === action.payload) state.currentProject = null
    },
    setCurrentProject(state, action: PayloadAction<Project | null>) {
      state.currentProject = action.payload
    },
    setStatusDate(state, action: PayloadAction<{ projectId: string; statusDate: string | null }>) {
      const { projectId, statusDate } = action.payload
      const project = state.projects.find(p => p.id === projectId)
      if (project) project.status_date = statusDate
      if (state.currentProject?.id === projectId) {
        state.currentProject.status_date = statusDate
      }
    },
    setLoading(state, action: PayloadAction<boolean>) {
      state.loading = action.payload
    },
    setError(state, action: PayloadAction<string | null>) {
      state.error = action.payload
    },
  },
})

export const {
  setProjects, addProject, updateProject, deleteProject,
  setCurrentProject, setStatusDate, setLoading, setError,
} = projectSlice.actions
export default projectSlice.reducer

```

// store/slices/tasksSlice.ts

```
import { createSlice, type PayloadAction } from '@reduxjs/toolkit'
import type { Task, Dependency } from '@/types'

interface Snapshot { tasks: Task[]; dependencies: Dependency[] }

interface TasksState {
  tasks: Task[]
  dependencies: Dependency[]
  clipboard: Task[]
  selectedIds: string[]
  loading: boolean
  error: string | null
  undoStack: Snapshot[]
  redoStack: Snapshot[]
```

```
}
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

// store/index.ts

// store/provider.tsx

// store/slices/authSlice.ts

// store/slices/projectSlice.ts

// store/slices/tasksSlice.ts

```

const initialState: TasksState = {
  tasks: [],
  dependencies: [],
  clipboard: [],
  selectedIds: [],
  loading: false,
  error: null,
  undoStack: [],
  redoStack: [],
}

const tasksSlice = createSlice({
  name: 'tasks',
  initialState,
  reducers: {
    setTasks(state, action: PayloadAction<{ tasks: Task[]; dependencies: Dependency[] }>) {
      state.tasks = action.payload.tasks
      state.dependencies = action.payload.dependencies
      state.undoStack = []
      state.redoStack = []
    },
    addTasks(state, action: PayloadAction<Task[]>) {
      state.tasks.push(...action.payload)
    },
    updateTasks(state, action: PayloadAction<Task[]>) {
      for (const updated of action.payload) {
        const idx = state.tasks.findIndex(t => t.id === updated.id)
        if (idx !== -1) state.tasks[idx] = updated
      }
    },
    deleteTasks(state, action: PayloadAction<string[]>) {
      state.tasks = state.tasks.filter(t => !action.payload.includes(t.id))
      state.selectedIds = state.selectedIds.filter(id => !action.payload.includes(id))
    },
    copyTasks(state, action: PayloadAction<string[]>) {
      state.clipboard = state.tasks.filter(t => action.payload.includes(t.id))
    },
    clearClipboard(state) {
      state.clipboard = []
    },
    setSelectedIds(state, action: PayloadAction<string[]>) {
      state.selectedIds = action.payload
    },
    setLoading(state, action: PayloadAction<boolean>) {
      state.loading = action.payload
    },
    setError(state, action: PayloadAction<string | null>) {
      state.error = action.payload
```

```
4806        },
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

// store/index.ts

// store/provider.tsx

// store/slices/authSlice.ts

// store/slices/projectSlice.ts

// store/slices/tasksSlice.ts

```
    addDependency(state, action: PayloadAction<Dependency>) {
      state.dependencies.push(action.payload)
    },
    removeDependency(state, action: PayloadAction<string>) {
      state.dependencies = state.dependencies.filter(d => d.id !== action.payload)
    },
    updateDependency(state, action: PayloadAction<{ id: string; type?: number; lag?: number }>) {
      const idx = state.dependencies.findIndex(d => d.id === action.payload.id)
      if (idx !== -1) {
        if (action.payload.type !== undefined) state.dependencies[idx].type = action.payload.type
        if (action.payload.lag  !== undefined) state.dependencies[idx].lag  = action.payload.lag
      }
    },
    clearTasks(state) {
      state.tasks = []
      state.dependencies = []
      state.clipboard = []
      state.selectedIds = []
      state.undoStack = []
      state.redoStack = []
    },
    // ── Undo / Redo ──────────────────────────────────────────────────
    saveSnapshot(state) {
      state.undoStack.push({ tasks: [...state.tasks], dependencies: [...state.dependencies] })
      if (state.undoStack.length > 50) state.undoStack.shift()
      state.redoStack = []
    },
    undo(state) {
      if (state.undoStack.length === 0) return
      state.redoStack.push({ tasks: [...state.tasks], dependencies: [...state.dependencies] })
      const prev = state.undoStack.pop()!
      state.tasks = prev.tasks
      state.dependencies = prev.dependencies
    },
    redo(state) {
      if (state.redoStack.length === 0) return
      state.undoStack.push({ tasks: [...state.tasks], dependencies: [...state.dependencies] })
      const next = state.redoStack.pop()!
      state.tasks = next.tasks
      state.dependencies = next.dependencies
    },
  },
})

export const {
  setTasks, addTasks, updateTasks, deleteTasks,
  copyTasks, clearClipboard, setSelectedIds,
  setLoading, setError, clearTasks,
  addDependency, removeDependency, updateDependency,
```

```
    saveSnapshot, undo, redo,
```

```
// app/api/auth/login/route.ts
```

```
// app/api/auth/logout/route.ts
```

```
// app/api/auth/register/route.ts
```

```
// app/api/dependencies/[projectId]/route.ts
```

```
// app/api/projects/[id]/route.ts
```

```
// app/api/projects/route.ts
```

```
// app/api/tasks/[projectId]/[taskId]/route.ts
```

```
// app/api/tasks/[projectId]/route.ts
```

```
// app/api/versions/[projectId]/[versionId]/route.ts
```

```
// app/api/versions/[projectId]/route.ts
```

```
// app/dashboard/page.tsx
```

```
// app/globals.css
```

```
// app/layout.tsx
```

```
// app/login/page.tsx
```

```
// app/page.tsx
```

```
// app/projects/[id]/page.tsx
```

```
// app/register/page.tsx
```

```
// components/GanttChart/EditTaskModal.tsx
```

```
// components/GanttChart/GanttChart.tsx
```

```
// components/GanttChart/GanttToolbar.tsx
```

```
// components/GanttChart/VersionPanel.tsx
```

```
// components/ProjectCard.tsx
```

```
// components/auth/LoginForm.tsx
```

```
// components/auth/RegisterForm.tsx
```

```
// lib/auth.ts
```

```
// lib/db.ts
```

```
// lib/middleware.ts
```

```
// lib/result.ts
```

```
// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

// store/index.ts

// store/provider.tsx

// store/slices/authSlice.ts

// store/slices/projectSlice.ts

// store/slices/tasksSlice.ts

```
} = tasksSlice.actions
export default tasksSlice.reducer

```

// types/index.ts

```
export interface User {
  id: string
  username: string
  email: string
  created_at: string
}

export interface Project {
  id: string
  user_id: string
  name: string
  start_date: string | null
  end_date: string | null
  status_date: string | null
  created_at: string
}

export interface Task {
  id: string
  project_id: string
  parent_id: string | null
  task_code: string
  name: string
  assignee: string | null
  start_date: string | null
  end_date: string | null
  duration: number | null
  duration_unit: string
  percent_done: number
  is_milestone: boolean
  note: string | null
  order_index: number
  is_deleted: boolean
  deleted_at: string | null
  created_at: string
  updated_at: string
}

export interface TaskLifecycleEvent {
  id: string
  task_id: string
  task_code: string
  event_type: string
  field_name: string | null
```

```
    old_value: string | null
    new_value: string | null
    description: string
```

```
// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

// components/GanttChart/EditTaskModal.tsx

// components/GanttChart/GanttChart.tsx

// components/GanttChart/GanttToolbar.tsx

// components/GanttChart/VersionPanel.tsx

// components/ProjectCard.tsx

// components/auth/LoginForm.tsx

// components/auth/RegisterForm.tsx

// lib/auth.ts

// lib/db.ts

// lib/middleware.ts

// lib/result.ts

// next-env.d.ts
```

// next.config.ts

// postcss.config.mjs

// scripts/check-users.ts

// scripts/migrate.ts

// scripts/seed-admin.ts

// scripts/seed-demo.ts

// store/hooks.ts

// store/index.ts

// store/provider.tsx

// store/slices/authSlice.ts

// store/slices/projectSlice.ts

// store/slices/tasksSlice.ts

// types/index.ts

```
  created_by_name: string | null
  created_at: string
}

export interface Dependency {
  id: string
  project_id: string
  from_task_id: string
  to_task_id: string
  type: number
  lag: number
}

export interface CreateTaskInput {
  name: string
  parent_id?: string | null
  start_date?: string | null
  end_date?: string | null
  duration?: number | null
  duration_unit?: string
  percent_done?: number
  is_milestone?: boolean
  note?: string | null
  order_index?: number
}

export interface UpdateTaskInput extends Partial<CreateTaskInput> {
  id: string
}

export interface ChangeLog {
  id: string
  change_type: string
  task_name: string | null
  field_name: string | null
  old_value: string | null
  new_value: string | null
  description: string
}

export interface ProjectVersion {
  id: string
  project_id: string
  version_number: number
  task_count: number
  created_by_name: string | null
  created_at: string
```

```
  changes: ChangeLog[]
}

```